

Hardware Security Gateway for IoT Devices

A hardware-enforced security boundary between untrusted IoT devices and the networks they join

Project Concept Document | October 2026

One-line summary

Instead of building one chip that fits every IoT device (not practical), build a standardized security gateway that sits between IoT devices and the network and provides hardware-backed identity, isolation, filtering, encrypted communication and monitoring, with no changes to the devices themselves.

1. Background and motivation

IoT devices (cameras, sensors, printers, access-control systems, industrial controllers) are now common in organizations. Most were designed for low cost and long life, not security. When one joins an organizational network, it becomes a potential entry point into everything else on that network.

Two related observations started this project:

- **IoT on the corporate network.** A poorly secured device on the same network as employee PCs and servers gives an attacker a foothold for lateral movement.
- **Laptops on untrusted Wi-Fi.** Organizations ban personal laptops for data-breach reasons, but even work laptops face risk on public Wi-Fi (evil twin access points, ARP spoofing, DNS tampering).

Existing defenses are mostly software installed on the device or the host. IoT devices often cannot run that software, which is why a hardware boundary placed outside the device is attractive.

Core principle: Do not trust the IoT device. Do not trust the network. Put a hardware-enforced security boundary between them.

2. Real-world IoT security problems

Problem	Why it matters	Hardware opportunity
Default / hardcoded credentials	Mass compromise (e.g. Mirai botnet used factory passwords on cameras and routers)	Hardware identity replaces passwords
No patch path / legacy devices	Devices run 5-10+ years with known vulnerabilities; replacement is expensive	Security wrapper around the unpatchable device
Flat networks	One compromised device can scan and reach servers, PCs, databases	Hardware network isolation, least-privilege rules
Shadow IoT	Unknown devices plugged in by staff; nobody monitors or patches them	Device discovery + authentication before network access
Weak or no encryption	Plaintext MQTT/HTTP/Telnet is trivial to sniff or tamper with	Hardware crypto + secure tunnel
Spoofed or cloned identity	Attacker clones a sensor identity and joins the network	Keys in a secure element, never exposed
Malicious or tampered firmware	Supply chain or physical tampering	Secure boot, signed firmware verification
Abnormal behavior unnoticed	Compromised device quietly scans or exfiltrates data	Traffic baselining and anomaly alerts
Untrusted Wi-Fi for laptops	Evil twin, MITM, ARP spoofing on public networks	USB secure network interface with tunnel

Real-world example: in a widely reported casino breach, attackers reached the internal network through a networked fish-tank thermometer. The device itself held no valuable data; it was simply a path in.

3. Why not a universal chip?

The first idea was one chip compatible with almost every IoT device. This is not realistic:

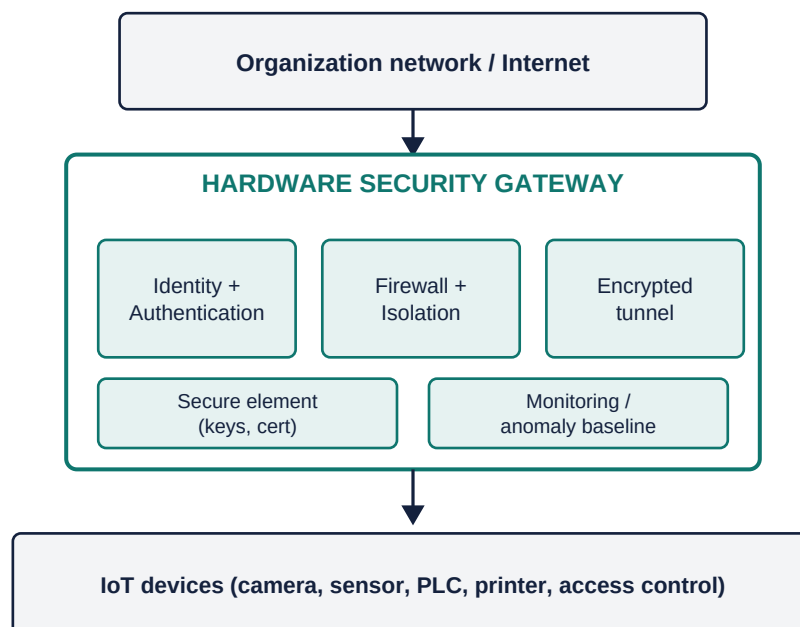
- IoT devices use different MCUs, buses (SPI, UART, I2C, CAN), power budgets, protocols (Wi-Fi, Zigbee, BLE, LoRa, Modbus) and operating systems.

- Embedding a chip means every vendor must redesign their product around it.
- Silicon fabrication is expensive, and mature secure elements (Microchip ATECC608, TPMs, ARM TrustZone) already exist.
- The research gap is not a better crypto chip. It is a way to protect existing and legacy devices *without* modifying them.

Reframed idea: Don't make every IoT device secure. Put a security boundary around the IoT environment.

4. Proposed solution: Hardware Security Gateway

A physical device with two sides. The device-facing side connects to IoT equipment (Ethernet first, optionally serial or Wi-Fi). The network-facing side connects to the organization's network. All IoT traffic passes through the gateway, which acts as a policy enforcement point.



4.1 Packet decision flow

Every packet from the IoT side is evaluated before it is allowed onto the network.



4.2 Core security functions

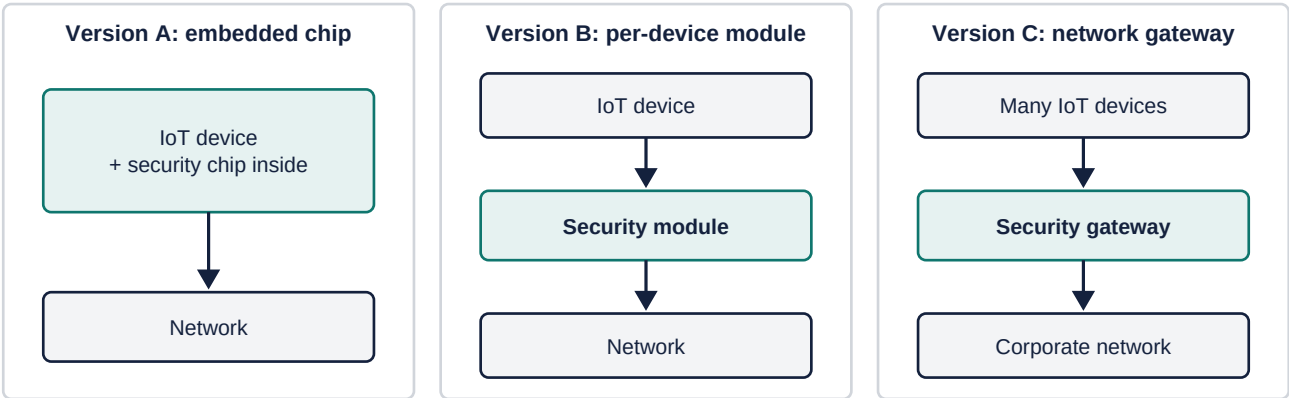
Function	What it does	Example
Hardware identity	Each gateway/device has a unique cryptographic identity stored in a secure element. Private key never leaves the chip.	Certificate + key for "Camera A"
Authentication	Challenge-response proves the device is who it claims to be before it gets network access.	Cloned or rogue device is rejected
Isolation / segmentation	Least-privilege allow-lists per device.	Camera may reach only 10.10.20.50:443
Firewall and filtering	Rules on source/destination IP, ports, protocol, MAC, connection state, rate.	Block SSH, DB access, port scans
Encryption / VPN	Gateway builds the tunnel on behalf of legacy devices that cannot.	WireGuard or IPsec to a security server
Secure storage	Keys and certificates in a secure element.	ATECC608 or similar
Monitoring	Learns a per-device baseline (destinations, ports, rates) and alerts on deviation.	Camera suddenly contacts 20 internal hosts on port 4444

4.3 Example: compromised camera

- **Without the gateway:** attacker takes over camera, port-scans the internal network, reaches file server and database.
- **With the gateway:** camera traffic is allowed only to the camera server. Scans, SSH attempts and unknown destinations are dropped and logged, and the anomaly raises an alert.

5. Where does it physically sit?

It does not have to be one device per IoT device. There are three deployment models:

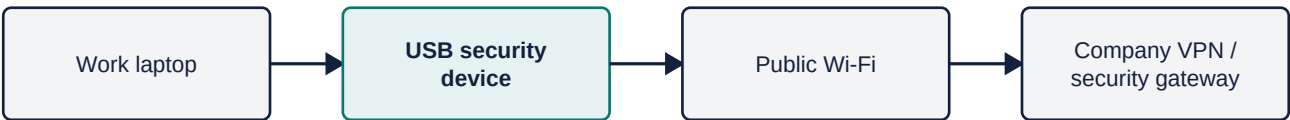


Version	Form	Best for	Trade-off
A: Embedded chip	Secure chip inside the IoT product	New products	Needs vendor adoption; not our starting point
B: Per-device module	Small box/dongle in front of one device	Legacy or high-value devices (e.g. PLC)	Cost and count scale with devices
C: Network gateway	One appliance per IoT segment/VLAN	Large fleets: cameras, sensors, printers	Protects the segment, not each device individually

Recommendation: prototype Version C (with Version B as a single-device form of the same design). One design covers many devices and is the easiest to demonstrate and evaluate. A large organization would deploy one gateway per security zone, for example office cameras, building sensors, industrial PLC network, and medical IoT.

6. Second mode: USB secure network device for laptops

The same architecture can protect a work laptop on untrusted Wi-Fi.



- The device presents itself to the laptop as a USB Ethernet adapter (RNDIS/CDC-ECM).
- It connects to the public Wi-Fi itself and builds an encrypted tunnel to the company gateway. The laptop never talks to the hostile network directly.
- It can also detect evil twin APs (same SSID, different BSSID), ARP spoofing and DNS tampering on the upstream network.
- It works on locked-down machines and any OS without installing software.

Honest framing: public Wi-Fi alone does not mean data is stolen. WPA2/3, TLS and VPNs already help. The value of this device is *hardware-enforced isolation* from the untrusted network plus attack detection, not just encryption.

7. Prototype plan

No chip fabrication is needed. A prototype can be built from off-the-shelf parts and later proposed as a dedicated SoC/ASIC.

Component	Prototype option
Processing	Raspberry Pi (CM4 / Zero 2 W) or an MCU + FPGA board
Network interfaces	Dual Ethernet (USB-Ethernet adapter on a Pi), Wi-Fi for USB mode
Secure element	Microchip ATECC608 / TPM module (I2C)
Firewall / filtering	nftables on Linux first, later hardware-accelerated filtering
Tunnel	WireGuard (lightweight, easy to evaluate)
Monitoring	Python collector + baseline rules, optional ML model later
Test targets	IP camera or ESP32 sensor, plus an attacker machine running nmap

Build in layers so the first prototype stays small:

- **Layer 1: Hardware identity** (who are you?) with secure element and certificate-based authentication.
- **Layer 2: Isolation** (what may you talk to?) with firewall allow-lists and segmentation.
- **Layer 3: Protected communication** (how is traffic protected?) with a crypto tunnel.
- **Layer 4: Monitoring and AI** as an optional later layer for anomaly detection. The isolation and authentication must stay hardware-enforced and not depend on the AI.

Suggested timeline

Phase	Weeks	Deliverable
1. Threat model and scenarios	1	8-10 IoT attack scenarios; pick the ones solvable at the hardware boundary
2. Basic gateway	2-3	Bridge/firewall between camera and network with allow-list rules
3. Identity + tunnel	4-5	Secure element auth, WireGuard tunnel for IoT traffic
4. Monitoring	6-7	Baseline and alerts for scans/new destinations
5. USB mode (optional)	8-9	Pi Zero USB gadget mode with tunnel and evil-twin detection
6. Evaluation + write-up	10	Measurements, demo video, report

8. Research question and evaluation

Research question: Can a low-cost, hardware-enforced security gateway provide device authentication, network isolation, access control and protected communication for heterogeneous and legacy IoT devices without modifying their firmware?

Experiments should compare the same attacks with and without the gateway:

- **Security:** unauthorized connections blocked, port scans stopped, lateral-movement attempts blocked, rogue device rejected, anomaly detection rate and false positives.
- **Performance:** added latency, throughput, authentication time, CPU/memory and power use.
- **Practicality:** compatibility with different device interfaces, unit cost, ease of deployment.

9. Existing solutions and the gap

The idea overlaps with known technologies, which is useful to cite and position against:

- Commercial IoT security platforms (Armis, Forescout, Palo Alto IoT Security) and enterprise NAC/segmentation products, mostly software or large appliances.
- Secure elements and TPMs for device identity; ARM TrustZone and secure boot inside devices.
- PUF-based identity research (device-unique keys derived from silicon variation).
- Standard VPN/ZTNA clients for laptops.

Possible gap: a low-cost, small-form-factor gateway that combines hardware identity, per-device isolation and tunneling for legacy devices, and that can be evaluated with measurable results. A PUF-based identity layer is a strong optional research extension.

10. Risks and open questions

- The gateway becomes a single point of failure; consider bypass/fail-closed behavior and redundancy.
- Encrypted device traffic limits deep inspection; most rules will use metadata (IP, port, protocol, volume).
- Wireless-only IoT protocols (Zigbee, BLE, LoRa) need a separate bridge; start with Ethernet/IP devices.
- Policy creation at scale: how are allow-lists learned and managed? A learning mode that proposes rules is worth building.

- The gateway itself must be hardened (secure boot, locked debug ports, signed updates, tamper resistance).

11. Next steps

- Write the threat model: list 8-10 concrete IoT attack scenarios and mark which the gateway can stop.
- Pick the prototype hardware and order parts (Pi, secure element, USB Ethernet adapter, test camera or ESP32).
- Build the minimum demo: camera behind the gateway, allow-list enforced, attacker blocked.
- Record before/after results; this becomes the core of the resume and research write-up.